```
}
```

We added a new static declaration of our known classes, `static classes = ["hidden"]`, and, alas, the associated TypeScript declaration `hiddenClass: string`. Stimulus creates a `hasHiddenClass: boolean` property that we're not using in this example. Then, in our `updateHiddenClass` method, we can replace the direct usage of `hidden` with the indirect reference to `this.hiddenClass` and everything works as before.

Tailwind and other utility CSS libraries generally require you to add or remove multiple classes, to better allow work with those libraries, Stimulus also creates a `hiddenClasses` plural attribute that is an array of strings, with each space-delimited class name an entry in the array.

Why would you use Stimulus classes since they seems to be making the code slightly more complex? There are a few reasons:

- The code is now decoupled from the CSS class name, so if the CSS class name changes, the code doesn't have to also change. This makes it harder to have a bug because you forgot to change the name somewhere.
- The CSS class name is now in the view, which is valuable if you have designers who don't want to deal with JavaScript working on the app.
- The CSS class name is now in the view, which means it's controlled by Rails. Which also means it's much easier to have a dynamic CSS class name, for example, if the CSS class name depended on internationalization or some other property of the Rails side of the site.

In addition to all of those reasons, being able to put all this data in the markup allows us to have generic, reusable Stimulus controllers. In much the same way Turbo allows us to add a lot of functionality without writing much custom JavaScript, writing generic Stimulus controllers can allow us to add a lot of functionality without adding a lot of custom JavaScript.